

Court of Public Opinion

Game Vision Execution Checklist

****Purpose:**** Convert the high-level game vision into an implementation sequence that an AI coding model can follow without drifting.

****Target stack:**** Vite + TypeScript + Three.js + HTML/CSS UI overlay + Colyseus-compatible room architecture

****Visual style:**** PS1 retro

****Core loop:**** Structured courtroom trial with solvable cases, evidence, objections, and jury verdicts

****Target match size:**** 2 to 6 players

****MVP priority:**** Fast browser play, AI-filled seats, one courtroom, deterministic case logic

Delivery Rules

- Every step should produce a visible result, a code artifact, or a testable behavior.
- Do not skip ahead when a dependency is incomplete.
- Keep all systems data-driven where possible.
- Prefer fixed scope over speculative systems.
- Ship one stable courtroom experience before adding variety.

Phase 0. Project framing and repo setup

Goals

- Establish a single source of truth for scope.
- Prepare the repo and folder conventions.
- Create a predictable working rhythm for phased implementation.

Checklist

- Phase 0, Step 1: Create a new repository for the game and define the project name.
- Phase 0, Step 2: Add a root `README.md` describing the pitch, MVP scope, stack, and local run commands.
- Phase 0, Step 3: Add a `docs` folder for design, implementation, and content schemas.
- Phase 0, Step 4: Add a `src` folder layout plan in the docs before writing game code.
- Phase 0, Step 5: Define core principles in `docs/project-rules.md`:
 - deterministic case logic
 - structured player input

- jury decides verdict
- judge is bounded
- one courtroom map for MVP
- Phase 0, Step 6: Add a `/data` folder for cases, evidence, witnesses, jurors, cards, and role configs.
- Phase 0, Step 7: Add a `/public` folder plan for textures, fonts, audio, and model exports.
- Phase 0, Step 8: Decide naming conventions for IDs, scene files, UI panels, and gameplay systems.
- Phase 0, Step 9: Create a `TODO\_IMPLEMENTATION.md` master file linking all phase docs.
- Phase 0, Step 10: Define Definition of Done for each feature:
 - code exists
 - behavior is testable
 - UI feedback exists
 - data schema is documented
 - no blocking console errors

Exit criteria

- Repo structure is stable.
- Scope rules are written down.
- Implementation phases are documented.

Phase 1. Core project bootstrap

Goals

- Stand up the runnable browser project.
- Confirm rendering, input, and hot reload.
- Create the foundation the rest of the game builds on.

Checklist

- Phase 1, Step 1: Initialize a Vite TypeScript project.
- Phase 1, Step 2: Install Three.js and supporting runtime dependencies.
- Phase 1, Step 3: Install development tooling:
 - ESLint
 - Prettier
 - TypeScript strict settings
- Phase 1, Step 4: Add npm scripts for dev, build, preview, lint, and typecheck.
- Phase 1, Step 5: Create the base `src/main.ts` entryptpoint.
- Phase 1, Step 6: Create a minimal application shell that mounts:
 - Three.js canvas

- UI overlay root
- debug HUD root

- Phase 1, Step 7: Confirm the project runs locally with no errors.
- Phase 1, Step 8: Add a neutral background scene to verify rendering pipeline.
- Phase 1, Step 9: Add a basic resize handler for the renderer and camera.
- Phase 1, Step 10: Commit a clean bootstrap checkpoint.

Exit criteria

- The game boots in browser.
- Canvas and UI can render together.
- Tooling and scripts are stable.

Phase 2. MVP vision lock

Goals

- Freeze what will and will not be in MVP.
- Translate the concept into concrete player-facing features.

Checklist

- Phase 2, Step 1: Write a one-page MVP statement in `/docs/mvp-scope.md`.
- Phase 2, Step 2: Confirm the only shipped map for MVP is a single courtroom.
- Phase 2, Step 3: Confirm the only mandatory deep roles are Prosecutor and Defense.
- Phase 2, Step 4: Confirm Judge is bounded and Jury decides verdict.
- Phase 2, Step 5: Confirm Witness and Juror are light roles for MVP.
- Phase 2, Step 6: Confirm all player inputs are structured, not freeform.
- Phase 2, Step 7: Confirm all shipped cases are predefined data, not live generated.
- Phase 2, Step 8: Set MVP target content counts:
 - courtroom scene: 1
 - cases: 3 to 5
 - evidence cards: 20 to 30
 - objections: 8 to 12
 - witness archetypes: 6 to 10
 - juror archetypes: 6 to 10
- Phase 2, Step 9: Write a “not in MVP” section to avoid scope creep.
- Phase 2, Step 10: Freeze the MVP until the first playable exists.

Exit criteria

- MVP is explicit.

- Non-MVP items are clearly excluded.
- The model has a bounded target.

Phase 3. Art direction and content production plan

Goals

- Lock the visual language early.
- Define what models, textures, and images are actually needed.

Checklist

- Phase 3, Step 1: Create an art asset list for the courtroom scene.
- Phase 3, Step 2: Create an art asset list for character models.
- Phase 3, Step 3: Create an art asset list for UI illustrations and evidence art.
- Phase 3, Step 4: Define PS1 style rules:
 - low poly silhouettes
 - limited texture resolution
 - slightly exaggerated proportions
 - strong readable poses
 - fixed cinematic camera framing
- Phase 3, Step 5: Decide character rig complexity for MVP.
- Phase 3, Step 6: Define texture naming and export rules.
- Phase 3, Step 7: Create a list of required courtroom props:
 - judge bench
 - prosecution desk
 - defense desk
 - witness stand
 - jury box
 - defendant seat
 - verdict signage
- Phase 3, Step 8: Create a list of required 2D UI assets:
 - role badges
 - card frames
 - evidence thumbnails
 - reaction icons
 - verdict stamps
- Phase 3, Step 9: Create a list of required sound effects:
 - gavel hit

- objection sting
- card play click
- witness reveal
- verdict sting

■ Phase 3, Step 10: Create an asset production board grouped by must-have and nice-to-have.

Exit criteria

- Asset scope is measurable.
- Style rules are clear.
- Content production can begin in parallel.

Phase 4. Production sequencing

Goals

- Establish the implementation order the AI should follow.
- Prevent random feature jumps.

Checklist

- Phase 4, Step 1: Build client shell before networking.
- Phase 4, Step 2: Build courtroom scene before content variety.
- Phase 4, Step 3: Build phase-based match state before UI polish.
- Phase 4, Step 4: Build cards and evidence system before full role variety.
- Phase 4, Step 5: Build AI seat-filling after human flow works locally.
- Phase 4, Step 6: Add online rooms only after offline flow is playable.
- Phase 4, Step 7: Add content expansion only after one full case is stable.
- Phase 4, Step 8: Add audio and camera polish after the trial loop is complete.
- Phase 4, Step 9: Add menu and onboarding after core play is understandable.
- Phase 4, Step 10: Run a stability pass before building extra cases.

Exit criteria

- Order of work is fixed.
- Dependencies are respected.
- The AI has a clear build path.

Phase 5. Milestone checkpoints

Checklist

- Milestone A: Bootable scene with Three.js canvas and UI overlay.
- Milestone B: Courtroom scene renders with placeholder assets.
- Milestone C: Local single-case trial flow works with debug controls.
- Milestone D: Prosecutor and Defense can play cards and evidence.
- Milestone E: Judge can rule in bounded windows.
- Milestone F: Jury can evaluate and vote.
- Milestone G: AI can fill empty seats.
- Milestone H: Multiplayer room flow works.
- Milestone I: At least three complete cases are playable.
- Milestone J: Browser-ready MVP build is stable and readable.

Cursor handoff rules

- Break implementation into tickets that can be finished in one focused coding pass.
- Never ask the model to build unrelated systems in one prompt.
- Require each coding pass to state:
 - what files it will touch
 - what behavior it adds
 - what assumptions it is making
 - how to verify the result
- Require each pass to update docs when data schemas or architecture change.
- Require visual verification screenshots for every substantial UI or scene change.

Suggested first implementation tickets

- Ticket 1: Bootstrap Vite TypeScript project with Three.js canvas and overlay root.
- Ticket 2: Create shared app layout and renderer bootstrap.
- Ticket 3: Add courtroom scene placeholder geometry and fixed cameras.
- Ticket 4: Add debug phase controller for a fake trial.
- Ticket 5: Add card panel and simple action selection UI.
- Ticket 6: Add JSON schema for one example case.
- Ticket 7: Load the example case and show role-specific data.
- Ticket 8: Add evidence panel and evidence reveal actions.
- Ticket 9: Add judge ruling window and jury vote screen.
- Ticket 10: Replace placeholder content with one complete playable trial.